

同濟大學

TONGJI UNIVERSITY

操作系统课程设计

课题名称	Unix V6++ 虚拟存储器页表机制改造 报告
副标题	按页框管理与进程私有页表设计
学院	计算机科学与技术学院
专业	计算机科学与技术
学生姓名	倪雨舒
学号	2353105
指导教师	操作系统课程设计
日期	2026 年 6 月 20 日

目 录

1	改造目标与总体思路	1
2	物理内存按页框管理	2
2.1	位图分配器	2
2.2	PageManager 的引用计数	3
3	进程私有页表与地址空间切换	5
3.1	页目录成为进程状态	5
3.2	每个进程只维护自己的私有用户页表	5
3.3	页目录的三类入口	6
3.4	上下文切换时刷新 CR3	7
4	进程生命周期中的页表维护	8
4.1	fork: 相同虚拟地址, 不同物理语义	8
4.2	exec: 重新按页装入进程镜像	8
4.3	brk 和栈增长	9
4.4	exit: 退出时逐页回收	9
4.5	内核复制页也要经过页表	10
5	运行验证	11
5.1	pageDemo 的验证思路	11
5.2	地址布局输出	12
5.3	fork 后的隔离结果	13
6	总结	14

1 改造目标与总体思路

这次课程设计的目标，是把 Unix V6++ 中原本偏向连续进程镜像的内存管理方式，往真正按页组织的虚拟存储器模型推进。原来的系统虽然已经打开了硬件分页，但是内核侧很多逻辑仍然更像是在维护一整段连续的用户空间：进程增长、复制和释放都比较粗粒度，页表更多只是最后映射时才会碰到的结构。

我这次主要做了几件事。第一，把用户物理内存拆成 4KB 页框，用位图记录哪些页框已经被占用；第二，让每个进程拥有自己的页目录和私有用户页表，而不是所有进程都依赖同一份运行时页表；第三，在 `fork()`、`exec()`、`brk()`、栈增长和 `exit()` 这些路径里都改成逐页维护映射；最后通过 `pageDemo` 验证父子进程可以拥有相同的虚拟地址布局，但写入互不污染。

整体设计可以概括成下面这条线：

- (1) 物理内存按页框分配和释放，不再只按连续空闲区来理解；
- (2) 每个物理页框增加引用计数，为共享页和后续写时复制留下接口；
- (3) 每个进程保存自己的页目录，页目录中同时挂接公共页表、内核页表和进程私有页表；
- (4) 上下文切换时同步刷新 CR3，让 CPU 真正切换到当前进程的地址空间；
- (5) 运行时通过 `pageDemo` 检查代码段、数据段、堆、栈的页布局，以及 `fork()` 后父子进程的隔离效果。

所以这次改造并不是单独换了一个分配器，而是把进程地址空间的组织方式整体改成了“虚拟页到物理页框”的模型。后面的章节会按这个顺序说明实现细节。

2 物理内存按页框管理

2.1 位图分配器

改造的第一步是把物理内存的管理单位落到页框上。新增的 BitMap 保存一个页框池的起始地址、位图行数和位图数组，每一位对应一个 4KB 页框的占用状态。

```

1  class BitMap {
2  public:
3      unsigned long m_AddressIdx;
4      int rows;
5      unsigned long long int map[256];
6
7      void set(unsigned long startAddr, unsigned long page_num) {
8          m_AddressIdx = startAddr;
9          rows = page_num / 64;
10         for (int i = 0; i < rows; i++) {
11             map[i] = 0;
12         }
13     }
14 };

```

代码 2.1 页框位图的核心结构

BitMapAllocator 的分配逻辑也比较直接：先把申请大小向上取整成页数，再扫描位图寻找一段足够长的空闲页框，找到后把对应的位设置为 1。释放时则根据物理地址反推页框下标，再把对应位清零。

```

1  unsigned long BitMapAllocator::Alloc(BitMap &bitmap, unsigned long
    size) {
2      int pages_needed = (size + M_PAGE_SIZE - 1) / M_PAGE_SIZE;
3      int start = -1, count = 0;
4
5      for (int i = 0; i < bitmap.rows * 64; i++) {
6          if (is_free(bitmap, i)) {
7              if (count == 0) start = i;
8              if (++count == pages_needed) break;
9          } else {
10             count = 0;
11         }
12     }
13
14     if (count < pages_needed) return 0;
15     for (int i = 0; i < pages_needed; i++) {

```

```

16         set_bit(bitmap, start + i, 1);
17     }
18     return start * M_PAGE_SIZE + bitmap.m_AddressIdx;
19 }

```

代码 2.2 按页框扫描和标记空闲位图

从这里开始，系统不再把用户物理空间只看成一整块连续内存，而是把它拆成很多页框去分配。这个变化看起来不大，但后面所有“同一个虚拟地址可以映射到不同物理页”的能力，都依赖这一步。

2.2 PageManager 的引用计数

PageManager 在位图之上又加了一层页引用计数。位图回答“这个页框有没有被占用”，而 Page[] 回答“这个页框现在被多少地方引用”。这两个问题不能混在一起，因为父子进程共享页时，一个页框虽然已占用，但不能因为某一个进程退出就立刻回收。

```

1  class PageManager {
2  public:
3      BitMap bitmap;
4      int Page[MemoryDescriptor::USER_SPACE_SIZE / PAGE_SIZE];
5
6      unsigned long AllocMemory(unsigned long size);
7      unsigned long FreeMemory(unsigned long size, unsigned long
          startAddress);
8  };

```

代码 2.3 PageManager 中的页框状态

实际分配和释放时，AllocMemory() 通过位图分配器拿到物理页框，并把引用计数初始化为 1；FreeMemory() 则先递减引用计数，只有引用计数归零时才真正清空位图。

```

1  unsigned long PageManager::AllocMemory(unsigned long size) {
2      unsigned long newaddr = this->m_pAllocator->Alloc(this->bitmap
          , size);
3      Page[newaddr >> 12] = 1;
4      return newaddr;
5  }
6
7  unsigned long PageManager::FreeMemory(unsigned long size,
8      unsigned long startAddress)
9      {
10     Page[startAddress >> 12]--;

```

```

10     if (Page[startAddress >> 12] == 0) {
11         this->m_pAllocator->Free(this->bitmap, size, startAddress)
12         ;
13     }
14     return 1;
15 }

```

代码 2.4 分配和释放时维护页引用计数

这样处理以后，页框的生命周期就清楚了：申请时变成被占用，复制或共享时增加引用，最后一个引用释放时才真正归还给分配器。这也为 `fork()` 中的共享页和写时复制语义打好了基础。

3 进程私有页表与地址空间切换

3.1 页目录成为进程状态

原来的地址空间管理更像是维护一份“当前正在使用的用户页表”，不同进程运行时再把自己的映射写进去。这次修改后，页目录被放进了 Process 结构，每个进程都保存自己的页目录指针。

```

1  class Process {
2  public:
3      unsigned long GetPageDirectoryPhyAddr();
4      PageDirectory *pPageDirectory;
5  };
6
7  unsigned long Process::GetPageDirectoryPhyAddr() {
8      return (unsigned long)(this->pPageDirectory)
9          - Machine::KERNEL_SPACE_START_ADDRESS;
10 }
    
```

代码 3.1 进程保存自己的页目录

这里有一个容易出错的地方：内核里保存的是页目录在高地址空间中的线性地址，而写入 CR3 的必须是物理地址，所以 GetPageDirectoryPhyAddr() 需要减去 KERNEL_SPACE_START_ADDRESS。如果这里直接把内核线性地址写进 CR3，页表切换就不会按预期工作。

3.2 每个进程只维护自己的私有用户页表

MemoryDescriptor::Initialize() 现在只为当前进程申请一张私有用户页表，不再一次性构造全局用户页表数组。这样，进程自己的正文段、数据段、堆和栈都可以写进这张私有页表。

```

1  void MemoryDescriptor::Initialize() {
2      KernelPageManager &kernelPageManager =
3          Kernel::Instance().GetKernelPageManager();
4
5      this->m_UserPageTableArray =
6          (PageTable *) (kernelPageManager.AllocMemory(sizeof(
7              PageTable))
8              + Machine::KERNEL_SPACE_START_ADDRESS);
    
```

```

8  }
9
10 void MemoryDescriptor::MapToPageTable() {
11     User &u = Kernel::Instance().GetUser();
12     if (u.u_MemoryDescriptor.m_UserPageTableArray == NULL) return;
13     FlushPageDirectory(u.u_procp->GetPageDirectoryPhyAddr());
14 }

```

代码 3.2 为进程分配私有用户页表

MapToPageTable() 的含义也随之改变了。它不再负责把映射复制到一张公共运行时页表里，而是刷新当前进程自己的页目录。也就是说，页表内容已经作为进程状态保存好了，CPU 只需要切到这份页目录即可。

3.3 页目录的三类入口

进程页目录由 AllocPageDirectory() 和 InitProcPageDirectory() 共同完成。前者从内核页池中申请一个页目录页，后者填入三个关键入口：第 0 项指向公共 0 号用户页表，内核高地址项指向内核页表，第 1 项指向当前进程的私有用户页表。

```

1  PageDirectory *ProcessManager::AllocPageDirectory() {
2      KernelPageManager &kernelPageManager =
3          Kernel::Instance().GetKernelPageManager();
4      return (PageDirectory *) (kernelPageManager.AllocMemory(
5          sizeof(PageDirectory)) + Machine::
6          KERNEL_SPACE_START_ADDRESS);
7  }
8
9  void ProcessManager::InitProcPageDirectory(Process *proc,
10                                             PageTable *
11                                             privateUsrPageTable)
12  {
13      PageDirectory *pd = proc->pPageDirectory;
14
15      pd->m_Entrys[0].m_Present = 1;
16      pd->m_Entrys[0].m_ReadWriter = 1;
17      pd->m_Entrys[0].m_UserSupervisor = 1;
18      pd->m_Entrys[0].m_PageTableBaseAddress =
19          Machine::USER_PAGE_TABLE_BASE_ADDRESS >> 12;
20
21      unsigned int kidx =
22          Machine::KERNEL_SPACE_START_ADDRESS / PageTable::
23              SIZE_PER_PAGETABLE_MAP;
24      pd->m_Entrys[kidx].m_Present = 1;
25      pd->m_Entrys[kidx].m_ReadWriter = 1;

```

```

22     pd->m_Entrys[kidx].m_UserSupervisor = 0;
23     pd->m_Entrys[kidx].m_PageTableBaseAddress =
24         Machine::KERNEL_PAGE_TABLE_BASE_ADDRESS >> 12;
25
26     unsigned long frame =
27         ((unsigned long)privateUsrPageTable
28          - Machine::KERNEL_SPACE_START_ADDRESS) >> 12;
29     pd->m_Entrys[1].m_PageTableBaseAddress = frame;
30     pd->m_Entrys[1].m_Present = 1;
31     pd->m_Entrys[1].m_ReadWriter = 1;
32     pd->m_Entrys[1].m_UserSupervisor = 1;
33 }

```

代码 3.3 进程页目录的关键入口

这个结构可以理解为“公共映射 + 内核映射 + 进程私有映射”。公共和内核部分保持共享，进程真正变化的部分放在自己的 1 号用户页表里。

3.4 上下文切换时刷新 CR3

分页系统是否真正成立，关键不只是页目录有没有建出来，还要看进程切换时 CPU 是否真的切到了另一套页目录。本次修改把 FlushPageDirectory 改成了带参数的宏，调用时传入当前进程页目录的物理地址。

```

1  #define FlushPageDirectory(pgTable) \
2      __asm__ __volatile__ (" movl %0, %%cr3" : : "r"(pgTable));
3
4  #define SwtchUStruct(p) \
5      Machine::Instance().GetKernelPageTable() \
6          .m_Entrys[Kernel::USER_PAGE_INDEX].m_PageBaseAddress = \
7          (p)->p_addr >> 12; \
8      FlushPageDirectory((p)->GetPageDirectoryPhyAddr());

```

代码 3.4 动态刷新 CR3

这样，调度器选中不同进程时，不只是恢复寄存器和内核栈，也会同步切换页目录。没有这一步，前面所有“每个进程一份页表”的结构都只是静态数据；有了这一步，它们才真正变成运行时地址空间。

4 进程生命周期中的页表维护

4.1 fork：相同虚拟地址，不同物理语义

fork() 是最能体现这次设计变化的路径。创建子进程时，系统会为子进程申请自己的私有页表和页目录，再把父进程页表内容复制过去。为了给写时复制做准备，父进程中原本可写的页会先被改成只读，并增加物理页框引用计数。

```

1  void ProcessManager::ModifyPageTable(UserPageManager &
    userPageManager,
2                                     PageTable *pgTable) {
3      for (unsigned int j = 0; j < PageTable::
        ENTRY_CNT_PER_PAGETABLE; j++) {
4          if (pgTable->m_Entrys[j].m_Present &&
5              pgTable->m_Entrys[j].m_ReadWriter) {
6              pgTable->m_Entrys[j].m_ReadWriter = 0;
7              userPageManager.Page[
8                  pgTable->m_Entrys[j].m_PageBaseAddress]++;
9          }
10     }
11 }
```

代码 4.1 fork 前把可写页改为共享只读页

这一步的意义是让父子进程先共享同一个物理页框，但不能直接写。后续如果发生写入，就可以根据缺页异常和引用计数分配新页，实现“虚拟地址相同，但物理页隔离”的效果。即使当前实现还没有把写时复制链路全部做得很完整，结构上已经具备了这个方向。

4.2 exec：重新按页装入进程镜像

exec() 不再适合用“扩展一整块进程镜像，然后整体搬移”的方式理解。现在的做法是根据可执行文件信息，分别为正文段、数据段和栈分配页框，然后把这些物理页框写入当前进程的私有用户页表。

这里的重点不是某一段具体代码，而是控制流发生了变化：装入程序时，内核不再只关心一段连续物理内存的起点和长度，而是逐页建立“虚拟页号到物理页框号”的关系。之后只要刷新当前进程页目录，新的用户态地址空间就生效了。

4.3 brk 和栈增长

堆和栈是运行过程中会继续变化的区域，因此它们也必须按页更新。栈增长时，系统会在栈方向多申请一页，设置对应的页表项，然后刷新页目录。

```

1  void Process::SStack() {
2      User &u = Kernel::Instance().GetUser();
3      UserPageManager &userPgMgr = Kernel::Instance().
        GetUserPageManager();
4      MemoryDescriptor &md = u.u_MemoryDescriptor;
5      PageTableEntry *entrys = (PageTableEntry *)md.
        m_UserPageTableArray;
6
7      unsigned int index = 1024 - (md.m_StackSize >> 12) - 1;
8      md.m_StackSize += 4096;
9
10     unsigned int phyFrame = userPgMgr.AllocMemory(4096);
11     entrys[index].m_Present = 1;
12     entrys[index].m_UserSupervisor = 1;
13     entrys[index].m_ReadWriter = 1;
14     entrys[index].m_PageBaseAddress = phyFrame >> 12;
15
16     md.MapToPageTable();
17 }

```

代码 4.2 栈增长时补充一个页表项

brk() 的处理也类似。堆扩张时为新增虚拟页分配物理页框并填入页表项；堆收缩时则清除页表项并释放对应页框。这样堆尾地址移动时，不再意味着一整段连续内存被搬动，而是页表项发生了变化。

4.4 exit: 退出时逐页回收

进程退出时同样不能只释放一整块进程镜像。当前实现会遍历数据段和栈相关的页表项，逐页调用 FreeMemory()，再清掉页表项，最后释放私有用户页表本身。

这和前面的引用计数逻辑是配套的：如果某个物理页框只被这个进程引用，它会被真正归还给位图分配器；如果它还被别的进程共享，就只减少引用计数，不会提前释放。

4.5 内核复制页也要经过页表

连 `Utility::CopySeg()` 这样的辅助函数也同步改造了。它不再假设要复制的页天然在某个连续线性地址里，而是临时借用内核页表中的两个表项，把源页和目标页映射进内核虚拟空间，然后执行一页大小的复制。

```

1  void Utility::CopySeg(unsigned long src, unsigned long des) {
2      User &u = Kernel::Instance().GetUser();
3      Process *current = u.u_procp;
4      PageTableEntry *pt = Machine::Instance().GetKernelPageTable().
        m_Entrys;
5
6      unsigned long oriEntry1 = pt[borrowedPTE].m_PageBaseAddress;
7      unsigned long oriEntry2 = pt[borrowedPTE + 1].
        m_PageBaseAddress;
8
9      pt[borrowedPTE].m_PageBaseAddress = src / PageManager::
        PAGE_SIZE;
10     pt[borrowedPTE + 1].m_PageBaseAddress = des / PageManager::
        PAGE_SIZE;
11     FlushPageDirectory(current->GetPageDirectoryPhyAddr());
12
13     unsigned char *srcVirtual = (unsigned char *)
14         (Machine::KERNEL_SPACE_START_ADDRESS
15          + borrowedPTE * PageManager::PAGE_SIZE);
16     unsigned char *desVirtual = (unsigned char *)
17         (Machine::KERNEL_SPACE_START_ADDRESS
18          + (borrowedPTE + 1) * PageManager::PAGE_SIZE);
19     Utility::MemCopy((unsigned long)srcVirtual,
20                     (unsigned long)desVirtual,
21                     PageManager::PAGE_SIZE);
22
23     pt[borrowedPTE].m_PageBaseAddress = oriEntry1;
24     pt[borrowedPTE + 1].m_PageBaseAddress = oriEntry2;
25     FlushPageDirectory(current->GetPageDirectoryPhyAddr());
26 }

```

代码 4.3 通过临时内核映射复制一页

这个修改说明分页模型已经不只是进程管理层的概念，连内核内部复制页内容时也必须显式处理页表映射。

5 运行验证

5.1 pageDemo 的验证思路

为了验证页表改造是否真的生效，我写了一个 pageDemo 程序。它先打印一个进程内部几个典型地址：函数地址、全局变量地址、栈变量地址、几次 malloc() 得到的堆地址，以及 sbrk(0) 返回的堆尾地址。这样可以直观看到代码段、数据段、堆和栈分别落在哪些虚拟页中。

之后程序执行 fork()。子进程会打印 g_data 和 heap_b 的地址和值，然后把它们改掉；父进程等待子进程退出后，再打印同样的地址和值。如果父子进程看到的虚拟地址相同，而父进程的值没有被子进程写坏，就说明页表隔离是成立的。

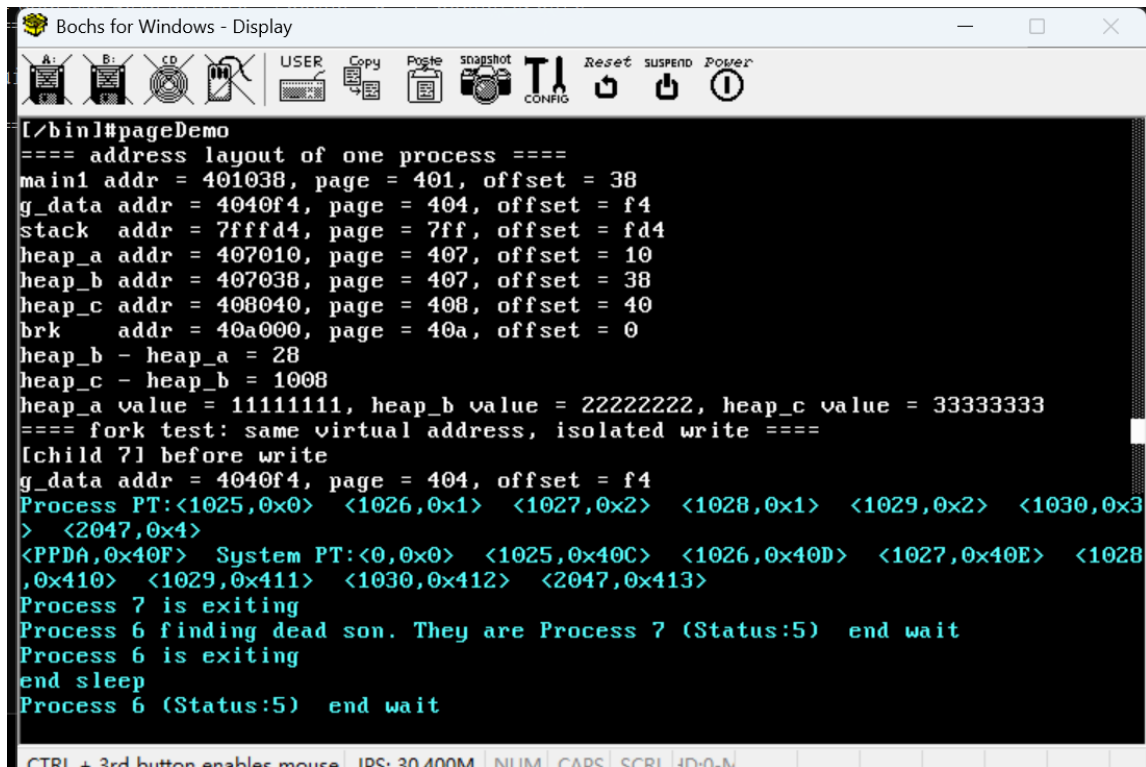
```

1  static int g_data = 0x12345678;
2
3  pid = fork();
4  if (pid == 0) {
5      printf("[child %d] before write\n", getpid());
6      print_block("g_data", (unsigned long)&g_data);
7      print_block("heap_b", (unsigned long)heap_b);
8      printf("[child %d] values before: g_data=%x heap_b=%x\n",
9             getpid(), g_data, *heap_b);
10
11     g_data = 0xcafebabe;
12     *heap_b = 0x0badc0de;
13
14     printf("[child %d] values after : g_data=%x heap_b=%x\n",
15            getpid(), g_data, *heap_b);
16     exit(0);
17 }
18
19 wait(&status);
20
21 printf("[parent %d] after child exit\n", getpid());
22 print_block("g_data", (unsigned long)&g_data);
23 print_block("heap_b", (unsigned long)heap_b);
24 printf("[parent %d] values now : g_data=%x heap_b=%x\n",
25        getpid(), g_data, *heap_b);

```

代码 5.1 pageDemo 中的 fork 验证逻辑

5.2 地址布局输出



```

Bochs for Windows - Display
[~/bin]#pageDemo
==== address layout of one process ====
main1 addr = 401038, page = 401, offset = 38
g_data addr = 4040f4, page = 404, offset = f4
stack addr = 7fffd4, page = 7ff, offset = fd4
heap_a addr = 407010, page = 407, offset = 10
heap_b addr = 407038, page = 407, offset = 38
heap_c addr = 408040, page = 408, offset = 40
brk addr = 40a000, page = 40a, offset = 0
heap_b - heap_a = 28
heap_c - heap_b = 1008
heap_a value = 11111111, heap_b value = 22222222, heap_c value = 33333333
==== fork test: same virtual address, isolated write ====
[child 7] before write
g_data addr = 4040f4, page = 404, offset = f4
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <1030,0x3>
> <2047,0x4>
<PPDA,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x40E> <1028,0x410>
<1029,0x411> <1030,0x412> <2047,0x413>
Process 7 is exiting
Process 6 finding dead son. They are Process 7 (Status:5) end wait
Process 6 is exiting
end sleep
Process 6 (Status:5) end wait
  
```

图 5.1 pageDemo 打印的进程地址布局和部分页表调试信息

从图 5.1 可以看到，main1 位于页 0x401，全局变量 g_data 位于页 0x404，栈变量在高地址附近的 0x7ff 页，堆从 0x407、0x408 继续增长，当前堆尾到了 0x40a000。这说明用户地址空间已经能按页观察：代码、数据、堆、栈分别落在不同逻辑页上，堆也能逐页扩张。

图中还打印了 Process PT 和 System PT。这些信息说明当前进程的用户页表内容确实参与了地址转换，而不是简单依赖一整块连续物理内存。

5.3 fork 后的隔离结果

```

heap_a_val = 1111111, heap_b_val = 2222222, heap_c_val = 3333333
=== fork test: same virtual address, isolated write ===
[child 7] before write
g_data addr = 4040f4, page = 404, offset = f4
heap_b addr = 407038, page = 407, offset = 38
[child 7] values before: g_data=12345678 heap_b=22222222
[child 7] values after : g_data=cafebabe heap_b=badc0de
[parent 6] after child exit
g_data addr = 4040f4, page = 404, offset = f4
heap_b addr = 407038, page = 407, offset = 38
[parent 6] values now : g_data=12345678 heap_b=22222222
[/bin]#

```

图 5.2 fork() 后父子进程同虚拟地址、写入隔离的结果

图 5.2 是最关键的结果。子进程和父进程看到的 g_data 地址都是 0x4040f4, heap_b 地址都是 0x407038, 说明父子进程保持了同构的用户虚拟地址布局。

但子进程把 g_data 改成 cafebabe、把 heap_b 改成 badc0de 后, 父进程再次读取时仍然得到原值 12345678 和 22222222。这就证明了当前页表设计的核心效果: 虚拟地址可以相同, 但底层物理页框映射已经被隔离, 子进程的写入不会污染父进程。

6 总结

这次页表改造的核心，是把 Unix V6++ 的用户内存管理从“连续进程镜像”的思路，推进到“按页框分配、按页表描述地址空间”的思路。实现上主要完成了下面几层变化：

- 物理页框由位图管理，PageManager 维护页引用计数；
- 每个进程有自己的页目录和私有用户页表；
- 页目录中同时保留公共用户页表、内核页表和进程私有页表；
- 上下文切换时动态写入 CR3，真正切换地址空间；
- `fork()`、`exec()`、`brk()`、栈增长和 `exit()` 都按页维护映射关系；
- `pageDemo` 验证了同一虚拟地址布局下的父子进程写入隔离。

从运行结果来看，现在系统已经具备了一个分页式地址空间应有的基本性质：进程看到的是自己的虚拟地址空间，物理页框由页表决定，父子进程可以拥有相同的虚拟地址，却不会因为一次写入互相破坏数据。

后续如果继续完善，可以把写时复制异常处理链路补完整，让只读共享页在第一次写入时自动分配新页；也可以继续清理旧的连续内存字段，让进程镜像描述彻底转向页表和页框数组。就当前课程设计目标来说，页框分配、私有页表、动态页目录切换和运行验证这几部分已经把核心思想串起来了。